

Lecture 5: Let's start our Projects

COSC 426 / 526: Introduction to Data Engineering



THE UNIVERSITY OF
TENNESSEE
KNOXVILLE
KNOXVILLE

BIG ORANGE. BIG IDEAS.®

Project Description

Analysis of Document Relationships Using Machine Learning and Artificial Intelligence

Project Description: Overview

- Analyzes intra-relationships within documents
 - Use advanced ML and AI technologies, including neural networks
- Examine inter-relationships among different documents
 - Identify connections via citations, shared themes, or section interactions
- Tackle challenges of diversity, scarcity, and inconsistent document formatting

Project Description: Steps

Goal: develop a straightforward solution to detect specific document intra- or/and inter-relationship types.

- Select a group of documents characterized by diverse, sparse, and inconsistently formatted content.
- Define the specific inter-relationships to be detected.
- Preprocess the raw data and model it to make it compatible with ML and AI techniques.
- Create and use a Jupyter Notebook to either
 - Classify (e.g., label relationships as “strong,” “weak,” or “none”)
 - Regress (e.g., predict a similarity score between two documents on a scale from 0 to 1) the relationships within the document set.
- Test the model on a document set and measuring its performance using quantitative success metrics.

Example of Relationships in Documents

- Relationships within a book: Ascertain the probability that individual pages belong to a specific chapter.
- Relationships among chapters in a Book: Evaluate whether different chapters are part of the same book.
- Relationships within university applications: Identify applications submitted for the same university program.
- Relationships among endorsement letters: Identify endorsement letters about the same individual.

Example of Relationships in Documents

- Relationships within company reports: Determine which sections or data points within a comprehensive annual report are related to financial performance.
- Relationships between meeting minutes: Link decisions recorded in the minutes of various departmental meetings to actions outlined in project management documents.
- Relationships within email threads: Trace the discussion regarding a particular, e.g., policy change or person within a threaded email conversation.
- Relationships among employee files: Identify common professional development trajectories by comparing multiple employee files or resumes.
- Relationships within contract documents: Connect addendums to their original contracts to ensure coherent interpretation and enforcement.

Modeling Raw Data

Model raw data into compatible formats for ML and AI techniques

Multiple techniques can be considered to transform text data into formats that can be processed by ML and AI models

- *Bag of Words (BoW)*
- *Word Embedding (WE)*
- *Character Embedding (CE)*

Bag of Words (BoW)

- BoW model: transform the text into numerical feature vectors
- Representation: Ignores grammar and order but considers word frequency
- Results in large, sparse vectors: Each unique word gets a distinct position
- Major limitation: Loses word order, which can affect syntactical meaning
 - Example issue: "dog bites man" vs. "man bites dog" illustrates sequence importance

BoW variation:
One hot
representation of
textual words

Vocabulary	What	is	behind	the	table	?
What	1	0	0	0	0	0
is	0	1	0	0	0	0
the	0	0	0	1	0	0
?	0	0	0	0	0	1
behind	0	0	1	0	0	0
left	0	0	0	0	0	0
chair	0	0	0	0	0	0
table	0	0	0	0	1	0

Bag of Words (BoW): n -grams model

BoW variation: a sequence of 'n' items from a given text

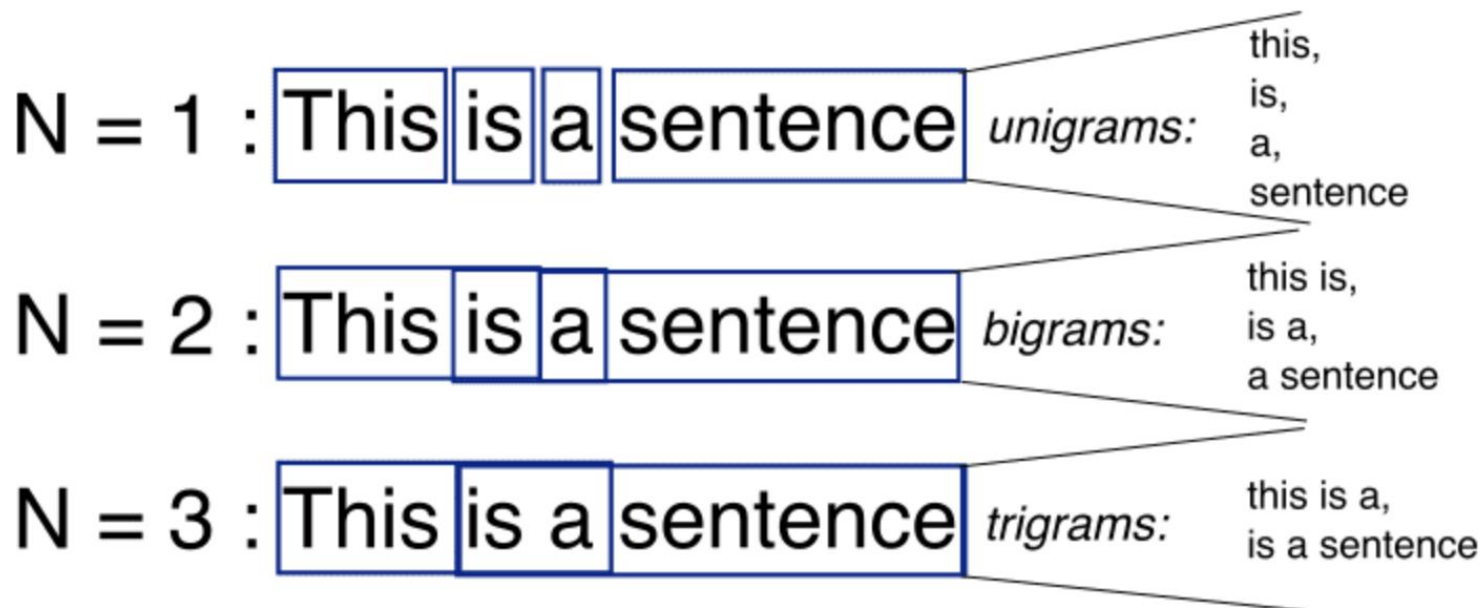
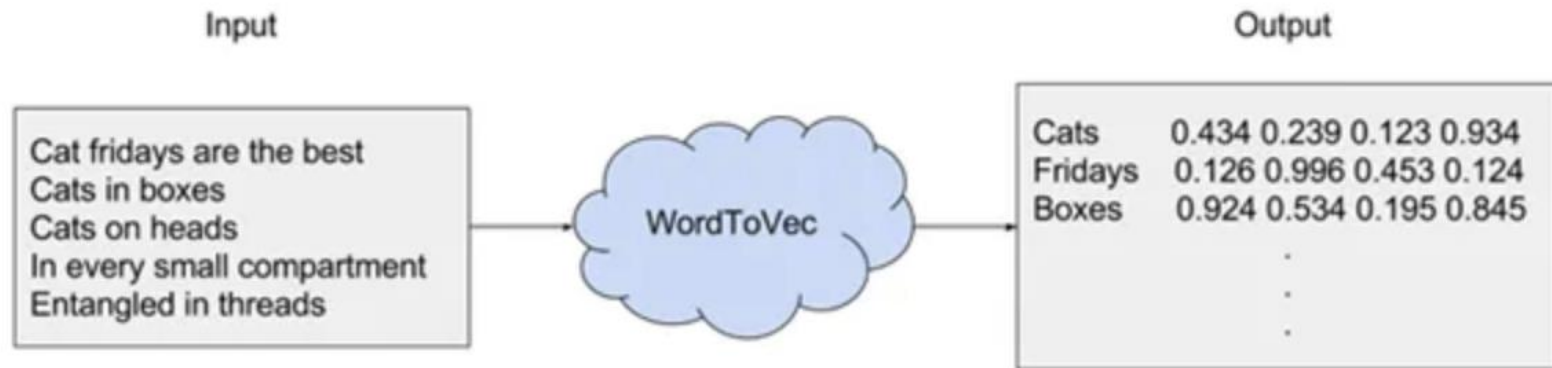


Figure 2 Uni-gram, Bi-gram, and Tri-gram Model

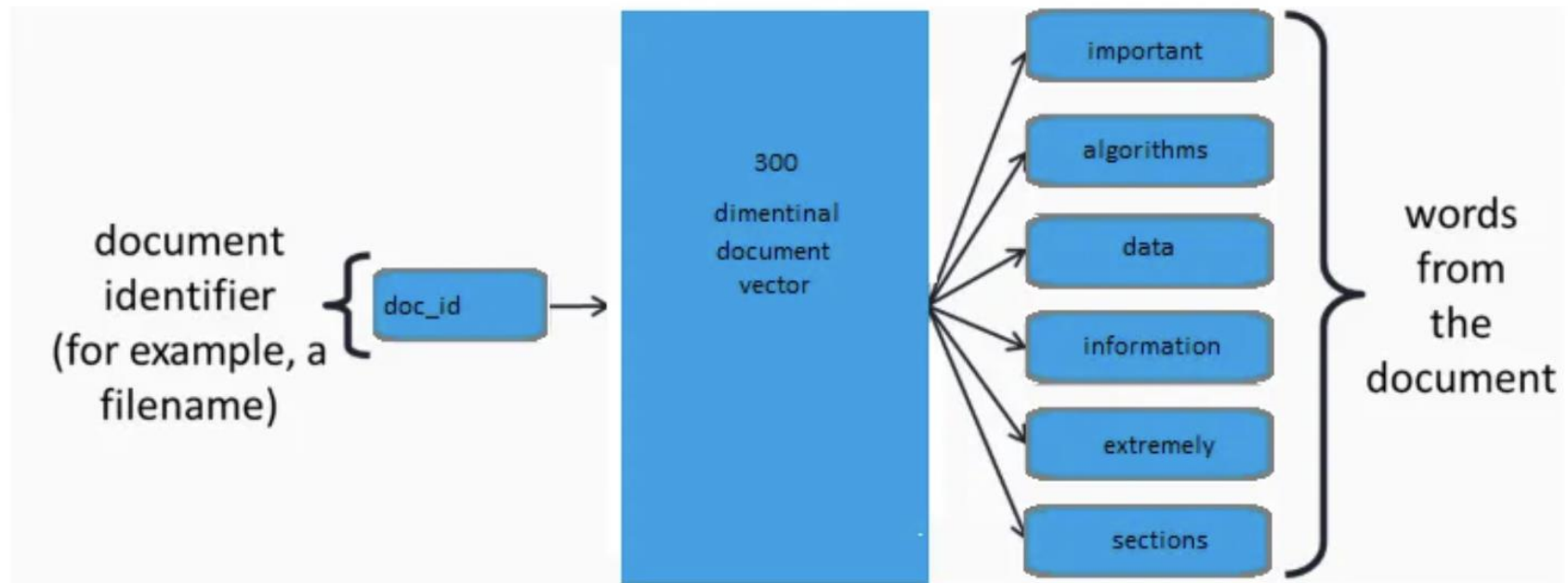
Word Embedding (WE)

- WE Model: Dense vector representation of words in low-dimensional space: use techniques like Word2Vec or GloVe
- Captures contextual meanings: Similar words have similar vector representations
- Preserves word order: Sequences maintained during model training
- Challenges: Struggles with word boundaries in whitespace-less texts or compound words
- Captures semantics: Encodes meanings and relationships between words
- Effective generalization: Allows models to apply learnings to new, unseen texts

Word Embedding (WE): Word2vec



Word Embedding (WE): Doc2vec



Word2Vec or Doc2Vec

For this project, choose between Word2Vec and Doc2Vec for preparing your raw data for ML and AI methods based on the specific nature of the documents you are working with and the kind of relationships you aim to analyze

Choose Word2Vec if you are:

- Analyzing the semantic similarity between words.
- Identifying topics based on word usage patterns.
- Discovering words used in similar contexts across different documents.

Choose Doc2Vec if you are:

- Comparing the thematic content of entire documents.
- Grouping documents by similarity in content.
- Tracking document changes or versions by comparing their embeddings.

Performing Classification or Regression

Select the ML or AI technique for the relationship identification

The ML or AI technique for classifying (e.g., label relationships as “strong,” “weak,” or “none”) or for regression (e.g., predict a similarity score between two documents on a scale from 0 to 1) the relationships within the document set is up to you.

Cosine similarity using Doc2Vec or Word2Vec

- Doc2Vec: By converting entire documents into vector representations, you can measure the cosine similarity between these vectors. A score of 1 indicates identical documents, and 0 indicates no similarity.
- Word2Vec: Although typically used for word-level comparisons, you can average or aggregate Word2Vec vectors of all words in two documents to create a document-level representation and then compute the cosine similarity.

Convolutional Neural Networks (CNNs) on Word2Vec vectors

Treating Word2Vec vectors like pixels in an image, you can employ 1D or 2D CNNs to detect patterns and learn higher-order features representing the documents. The CNN can be trained to output a similarity score directly, or its features can be used to calculate similarity through another metric.

ML models for regression

You can train a regression model (like a Random Forest or Support Vector Regression) on features derived from the Word2Vec. The model must be trained to predict a continuous similarity score between 0 and 1.

Phase 1: Selection of document set and definition of relationships

Tasks:

- Select a diverse group of documents with sparse and inconsistently formatted content.
- Clearly define the specific intra- and inter-relationship(s) to be detected within and among these documents.

Intermediate deliverables:

- A report outlining the chosen document set and the defined relationships

Phase 2: Preprocessing and modeling data for ML and AI Techniques

Tasks:

- Preprocess the raw data from selected documents.
- Apply data transformation techniques like Bag of Words, TF-IDF, or Word Embedding to model the data for ML/AI compatibility.

Intermediate deliverables:

- Processed and transformed datasets ready for analysis.
- An initial draft of Jupyter Notebook documenting data preparation steps.

Phase 3: Development of ML/AI models and initial testing

Tasks:

- Develop ML models to classify or regress the defined relationships in the document set.
- Test the models using a subset of the document set.

Intermediate deliverables:

- A fully developed Jupyter Notebook with ML models.
- Preliminary test results and performance metrics of the models.

Phase 4: Final testing, evaluation, and documentation

Tasks:

- Conduct comprehensive testing of the ML models on the entire document set.
- Measure the performance using quantitative success metrics.
- Prepare a project poster and an extended abstract detailing the project.

Intermediate deliverables:

- Finalized Jupyter Notebook with documented analysis and results.
- A poster summarizing the project.
- A two-page extended abstract providing a detailed account of the project.

Project Deliverables

Your project should be documented and structured in the following manner:

Dataset: Upload and store your dataset on Dataverse or another open-access platform for hosting research data.

Jupyter Notebook: Create a Jupyter Notebook that contains your analysis and solution. This notebook should be designed to access and use the dataset directly from Dataverse or the alternative platform where your data resides.

GitHub repository: Set up your project GitHub repository (we will provide you with it) to store your project's materials. This should include:

- A README file that provides an overview and instructions for navigating your project.
- The Jupyter Notebook containing your analytical work.
- Any additional code or scripts that are part of your project.

Project Deliverables

Poster: Design a poster that summarizes the problem you are addressing, the methods you have implemented, the results you have achieved, and potential future work in this area.

Extended abstract: Write a two-page extended abstract giving a detailed account of your project, including the problem statement, methodology, results, and suggestions for future research.

Project requirements & Key steps to succeed in your project

Project: Let's start ..

- Project deliverables: Jupyter notebook + Poster + Extended abstract (2 pages)
- The software has to be uploaded to a dedicated repository
 - We will generate the repository for you
- The data must reside in DataVerse
- Each project is a story you tell through deliverables
 - Generating a set of slides can help to tell your story

Tell a Story

- Generate a set of slides with your story
- Motivation:
 - One slide with the motivation for your work
- Contributions:
 - List up to three project contributions
 - We propose ...
 - We design
 - We implement ...
 - We demonstrate ...
 - We measure ...

Tell a Story (II)

- Background:
 - One or two slides with backgrounds (everything that is not your contributions but the work of others that can support the understanding of your work)
- Methodology:
 - Two or three slides with the overview of the methodology you use in your project to deliver the contributions, e.g.,
 - Did you define a new algorithm?
 - Did you implement a new library?
 - Did you deliver a new interface?
 - Describe the methodology, its components, and its features.

Tell a Story (III)

- Results:
 - Two or three slides with the results of your projects
 - What are your metrics of success?
 - How did you measure the metrics?
 - Pay attention to the visualization of your measurements
 - One figure one key message
 - Multiple figures can be used to compare and contrast different scenarios
- Conclusion slide with lessons learned and future work
 - Up to three key observations --- add up to three numbers from your results that are impressive
 - Up to two future steps – no project is ever completed; what would you do next if you had more time?

Example of Projects

Project: Let's start ..

- Project: Jupyter notebook + Poster + Extended abstract (2 pages)
- The material has to be uploaded to a dedicated repository
 - We will generate the repository for you
- Examples:
 - Dylan's project was on a dataset from the United States Federal Railroad Administration Office of Safety Analysis
 - Mike's project was on National Health and Nutrition Examination Survey (NHANES) dataset



Leveraging Spark and Docker for Scalable, Reproducible Analysis of Railroad Defects

Dylan Chapp and Surya Kasturi
Advisors: Michela Tauber and Nii Attoh-Okine

Motivation

- Railroad network resiliency depends on identification of defects
- Sensor-equipped monitoring cars collect rail and track-geometry data
- Can we use the data to predict defect occurrences in rail subdivisions?

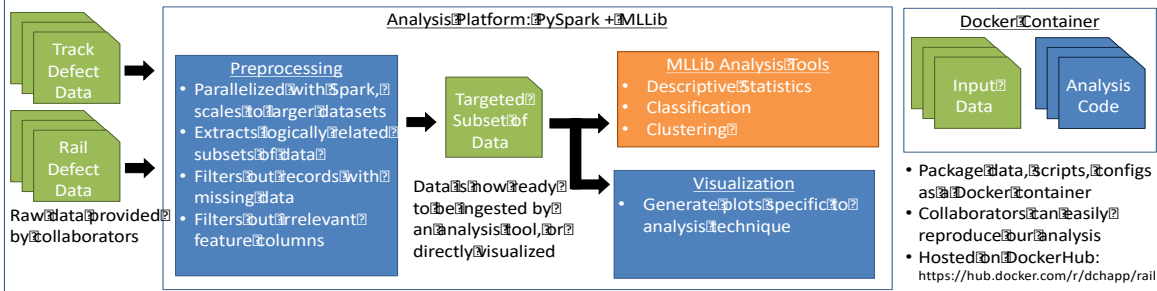


Rail and Track Defects Data Sets



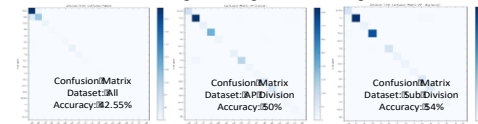
- Rail Defects Data: Physical degradation of 26,432 20-dimensional data points
- Track Geometry Data: Misalignment of 25,421 31-dimensional data points
- Mixed numerical and categorical data

Scalable Reproducible Data Analysis Workflow

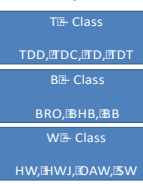


Predicting Defect Types

- Can we predict defects in railroad tracks?
- Defects are classified using Decision Tree
- Defect Size, Accumulated Tonnage, Rail Weight, Rail Section Age are used as features



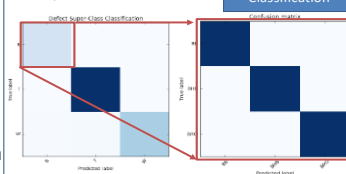
Defect Super Classes



Defect Super Class T, B, W Classification

Pipeline
Defects are classified using super-class before predicting their type

Using the subset containing T, B, W defects resulted in accuracy of 89.90%



Conclusions:

- We improved prediction accuracy by a hierarchical classification scheme
- First decide membership in defect super class, then in defect class using a second classifier

Track Region Similarity Analysis

- Can track regions be grouped so that defect-type classifiers trained on region-specific data achieve better accuracy?

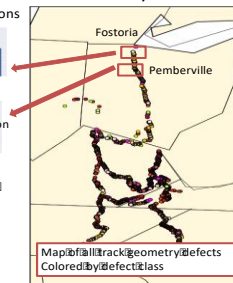
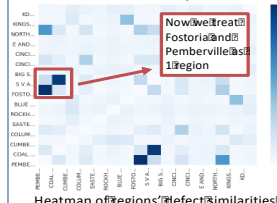
1. Extract each pair of regions' Defect Distributions



2. Compute the Chi-Squared Similarity of each pair of defect distributions

$$d(\mathbf{x}_u, \mathbf{x}_v) = \frac{1}{2} \sum_{n=1}^N \frac{[x_u(n) - x_v(n)]^2}{x_u(n) + x_v(n)}$$

3. Group regions together whose defect distributions are sufficiently similar



In progress:

- Training classifiers on subsets of defect data from statistically similar regions

References

- A. Zaremski, "Some Examples of Big Data in Railroad Engineering," IEEE International Conference on Big Data, 2014
- Track Inspector Rail Defect Reference Manual, Federal Railroad Administration, Rev. 2, 2015

Leveraging Spark and Docker for Scalable, Reproducible Analysis of Railroad Defects

Dylan Chapp
University of Delaware
dchapp@udel.edu

Surya Kasturi
University of Delaware
suryak@udel.edu

ABSTRACT

The resiliency of railroad networks depends on the ability of railroad engineers to identify and mitigate track and rail defects. As railroads modernize their defect identification measures, the volume and velocity of defect data substantially increases, necessitating adoption of techniques for "Big Data" analytics. We present a study of railroad defect prediction built atop Apache Spark and Docker to achieve scalability and reproducibility.

1. INTRODUCTION

According to the United States Federal Railroad Administration Office of Safety Analysis, track defects are the second leading cause of accidents on railways in the United States. In light of the economic significance of railway accidents [1], there is a pressing need in the railroad engineering community to adopt data-driven scalable data analysis tools from the greater "Big Data" ecosystem. [3] Track maintenance, i.e., identifying and repairing defects is one of the primary factors that affect the service life of a rail track, but due to the severe safety implications of undetected or unreported defects, the ability to predict common defects is highly desirable.

In this work, we present a case study centered on the analysis of two railroad defect data sets obtained from railroad engineering researchers in the University of Delaware Department of Civil Engineering. Hereafter we will refer to these datasets as the *rail_defects* data set and the *track_geometry_defects* data set. Respectively, these data sets describe defects in the rails themselves, such as voids or internal changes in crystalline structure, and misalignment of track components, such as one rail tilting away from the other. [3] We investigate the feasibility of predicting the type of a defect based on associated data such as geographic region, mean gross tonnage (MGT) the track is subject to, and rail type. In the rest of this paper, we outline the construction of our analysis platform, present some initial results on classification accuracy, and propose extensions to our work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2016 ACM. ISBN 978-1-4503-2138-9.
DOI: 10.1145/1235

2. METHODOLOGY

Both of the data sets we target have mixed categorical and numerical features and > 99% of the individual defect records have a class label indicating the type of defect. In the case of *rail_defects*, there are 20 distinct defect types. For *track_geometry_defects*, there are 25 defect types. In light of these properties, we focus on the multilabel classification task for each data set. We decompose the task into a pipeline of three parts: preprocessing, training, and testing. We implement this pipeline using the MapReduce framework Apache Spark [2] and its parallel machine learning library MLLib, and package the data and analysis scripts as a Docker container for ease of dissemination. In the remainder of this section, we describe the pipeline components, also display in Figure 1

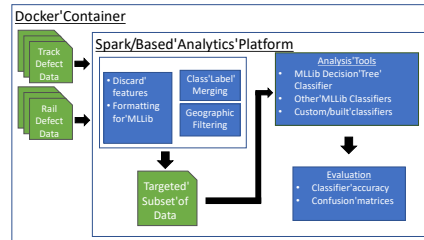


Figure 1: Block Diagram of Analytics Platform

We consider two stages of mandatory preprocessing. The first stage discards all columns except for a specified set, then discards any rows that are missing values for features from that set. The second stage maps the raw record strings to the format the MLLib API specifies, a key-value pair whose key is the type of defect and whose value is a feature vector. In addition to the above preprocessing, we implemented two optional stages: one to restrict the data to a geographically coherent region, and another to map each data point's class label to a "super-class" label indicating the general kind of defect (e.g., a welding-related defect, rather than one of the five kinds of specific welding defects). In our evaluation section, we demonstrate the usefulness of these additional preprocessing stages.

To build and evaluate our classifier, we split the subset of data remaining after preprocessing into training and testing

Table 1: Rail Defects Mapping

Super Class	Defect Types
T	TDD, TDC, TD, TDT
B	BRO, BHB, BB
W	HW, HWJ, OAW, SW
Others	SD, VSH, HSH, TW, CH, FH, PIPE, DR, EFBW

sets consisting of, respectively, 70% and 30% of the original data. Membership in the training and testing sets is determined by uniform random sampling. We then train an instance of MLLib's decision tree classifier on the training set and test its predictions. In principle, any other MLLib classifier with a compatible API could be trained instead, but we elected to keep our classifier type fixed and investigate the effect of the "class-merging" and "geographic filtering" preprocessing steps on accuracy.

3. EVALUATION

To evaluate our classifier's performance, we examine the overall accuracy rate of the classifier and its associated confusion matrix. When we trained the classifier on training data drawn uniformly at random from *rail_defects* dataset with each defect type as a class label, the classifier predicted with an accuracy of 42.55%.

3.1 Class Label Merging

We propose mapping each data point's class label to a "super-class" label indicating its general kind. Out of 20 defect types in *rail_defects* dataset, 11 are mapped to 3 three "super-classes". Table 3.1 shows mapped and unmapped defect types.

With this mapping, we show that the prediction accuracy of rail defects is improved using a hierarchical classification scheme. First a classifier is trained to decide super-class of data point, then a second classifier is used to predict its defect type. When this model applied on the training data, the classifier predicted with an accuracy of 89.90%. Figure 2 shows the confusion matrix of the respective result.

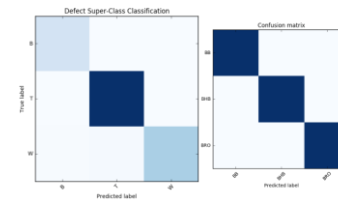


Figure 2: Confusion Matrix of the hierarchical classification scheme

3.2 Geographic Filtering

We propose that if subdivisions have similar numbers of each kind of defect, then we should group these subdivisions' data points and train a classifier with the expressed purpose of achieving good accuracy for that set of subdivisions. To

determine which subdivisions to merge, we propose computing the χ -squared distance S defined below for each pair of subdivisions, then merge them based on a fixed threshold.

$$S(D_1, D_2) = \sum_{i=0}^p \frac{(x_i - y_i)^2}{(x_i + y_i)}$$

We demonstrate the potential of grouping based on defect type distributions below. We compute $S(x, y)$ for each pair of subdivisions within the Appalachian division and display the results in the heat map in Figure 3.2

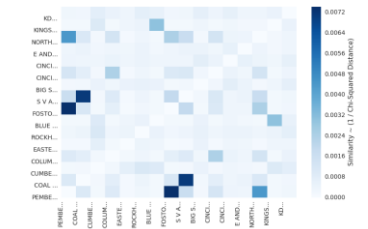


Figure 3: χ -squared distance between defect distributions for each subdivision

4. CONCLUSIONS AND CONTINUING WORK

We identified the potential of a hierarchical classification stage to improve the accuracy of defect type predictions. Additionally, we determined while that merely training classifiers on data from geographically-similar regions does not yield a significant improvement in accuracy, attempting to group together regions whose defect distributions are similar may prove useful.

Future directions for this work include evaluating classifiers beyond decision trees, and refining the similarity metric on defect distributions we use to group regions.

5. REFERENCES

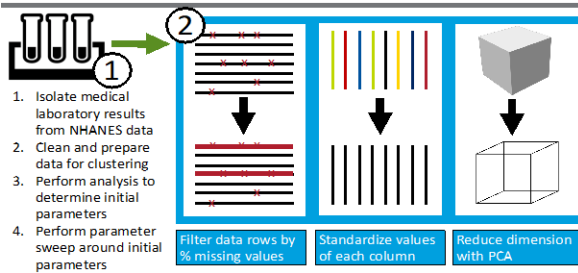
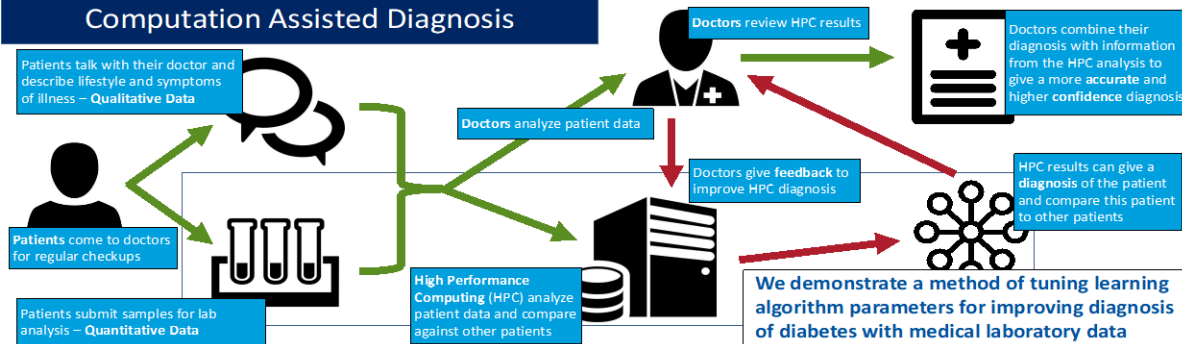
- [1] D. H. Schafer. A prediction model for broken rails and an analysis of their economic impact. *2008 AREMA Conference*, 2008.
- [2] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation*, NSDI'12, pages 2-2, Berkeley, CA, USA, 2012. USENIX Association.
- [3] M. A. Zaremski. Some examples of big data in railroad engineering. In *2014 IEEE International Conference on Big Data (Big Data)*, pages 96-102, Oct. 2014.



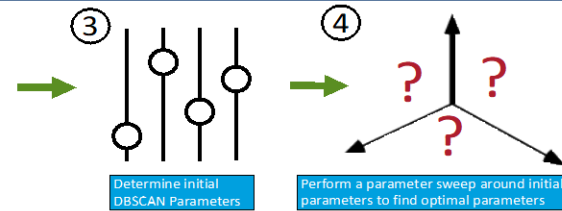
Parameter Tuning of DBSCAN for Medical Data and Diabetes Diagnosis

Michael Wyatt, Michela Taufer
University of Delaware: Global Computing Lab

Computation Assisted Diagnosis



Parameter Tuning Workflow



Parameter Tuning Results

- Missing data trade-off
 - No Missing Values ☐ Small Dataset
 - Many Missing Values ☐ Bad Clusters

Must find a balance between missing values and dataset size

- Patients with > 80% lab data available
 - 16,627 patients (over 1/5 original data)
 - Produces higher quality clusters

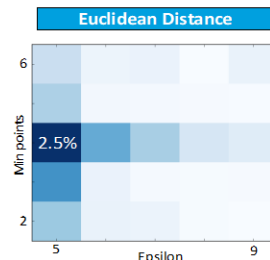
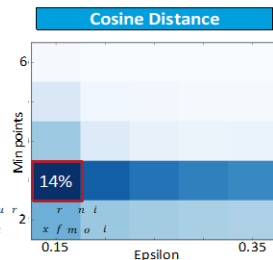
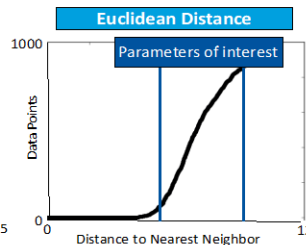
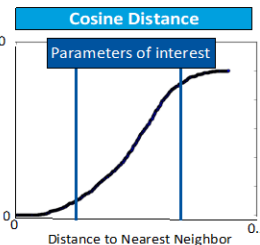
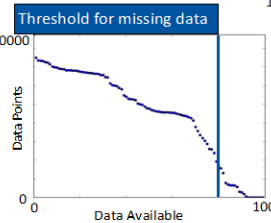
- DBSCAN parameters affect cluster quality
 - Epsilon**: Neighborhood to search for neighbors
 - Min_pts**: minimum neighbors to be in a cluster
 Parameters define cluster density
- Distance metrics also affect cluster quality: **Euclidean** vs. **Cosine**

- Utilizing nearest neighbor analysis, we can determine the range of epsilon values which should be tested
- We cluster data with several **epsilon** and **min_pts** values around the identified optimal values

- We measure the quality of each clustering by **percentage of points clustered** and **information gained** by each clustering:

$$score = \frac{Distance\ Points\ Clustered}{Total\ Distance\ Points} * \frac{Information\ Gained}{Total\ Information\ Gained}$$

- We identify an optimal parameter setting:
 - Cosine distance, Epsilon: 0.15, Min_pts: 3**



Parameter Tuning of DBSCAN for Medical Data and Diabetes Diagnosis

[Extended Abstract]

Michael R. Wyatt II
University of Delaware
18 Amstel Ave
Newark, DE 19701
mw Wyatt@udel.edu

Michela Taufer
University of Delaware
18 Amstel Ave
Newark, DE 19701
taufer@udel.edu

ABSTRACT

The increasing use of computationally assisted diagnosis in the doctor's office requires that computer diagnosis be both fast and accurate. We present a scalable method for preparing laboratory data for use with learning algorithms and a method for identifying optimal parameter settings for learning algorithms. To demonstrate our method, we predict the presence of diabetes among participants of the National Health and Nutrition Examination Survey using collected laboratory data and the DBSCAN algorithm. We performed optimization of the DBSCAN parameters for this dataset to demonstrate how diagnosis predictions can be improved.

CCS Concepts

•Applied computing! Health care information systems; Consumer health; •Computing methodologies! MapReduce algorithms;

Keywords

Health Informatics; Machine Learning; Optimization

1. MOTIVATION

Modern medical diagnosis is becoming increasingly computationally assisted. This means that artificial intelligence and machine learning algorithms are being used to analyze patient medical data and provide a diagnosis. Human doctors consult the computation results and a final diagnosis is made. As computationally assisted diagnosis becomes more widespread, patient diagnosis becomes more accurate [1] [3]. An important aspect of computationally assisted diagnosis is the processing of medical data by learning algorithms to produce useful results. Improving the speed and accuracy of this process will encourage the continued adoption of computationally assisted diagnosis by medical doctors, which will lead to improved population health and disease management.

2. CONTRIBUTIONS

Many efforts have been made to improve both the accuracy and processing time of computationally assisted diagnosis. These efforts focus mainly on the application of different algorithms to medical data. In this paper, we apply the clustering algorithm DBSCAN to medical data in order to diagnose patients with diabetes. We present a framework for

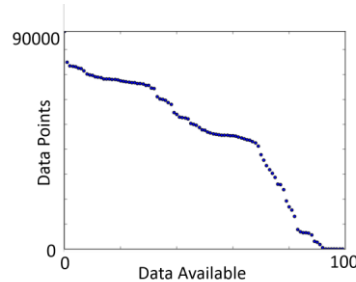


Figure 1: NHANES participants sorted by percent of laboratory data available.

parallel processing of medical data for learning algorithms. Additionally, we outline a method for optimizing learning algorithm parameters to achieve optimal results.

3. METHODOLOGY

We isolate lab data from the National Health and Nutrition Examination Survey (NHANES) dataset for over 70,000 participants. We process this data using parallel algorithms built with the MapReduce programming paradigm via Apache Spark. The processing of data is highly scalable across many nodes. The processed data is in a form that is usable by learning algorithms like DBSCAN. We then perform parameter optimization to achieve maximum predictive capabilities.

3.1 NHANES Dataset

We obtained data from the NHANES continuous dataset collected between 1999 and 2014. We label the participants as having or not having diabetes based on their categorical response to the question, "have you ever been told by a doctor or health professional that you have diabetes or sugar diabetes?" We isolate 116 common features within the laboratory data, including urine and blood sample values, which can be used to cluster the diabetic and non-diabetic NHANES participants.

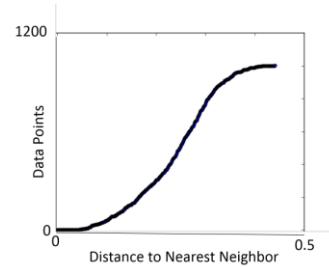


Figure 2: Number of data points (y axis) with a neighbor within distance $Epsilon$ (x axis). The elbow in the plot indicates optimal values for $Epsilon$.

3.2 Data Preparation

We prepare the data by removing participants with many missing values, standardizing the data by feature, and dimensionality reduction. Figure 1 plots the participants sorted by the percent of data points (from the 116 features) available. All NHANES participants had missing values and many had most laboratory values missing. We identify a subset of 16,627 participants with more than 80% of features available for analysis. Each of the 116 features are standardized using Z-score standardization. This process makes the range of values for each feature similar to prevent one feature outweighing others (due to a large range of values). Dimensionality reduction is performed with Principal Component Analysis (PCA). The processing of NHANES data is performed with MapReduce algorithms. This allows the process to be distributed across many compute nodes and reduces result turn-around time.

3.3 Choosing Initial Parameters

There are three parameters for DBSCAN. Together, they define the density of clusters which will be found.

1. $Distance$ - Metric used for calculating distance between patients
2. $Epsilon$ - Distance around patients to identify neighboring patients
3. Min_pts - Number of neighboring patients to be "core" point

Like other learning algorithms, these parameters affect the quality of results. We chose to test two distance metrics: Euclidean and Cosine. We define cosine distance in equation 1. A range of Min_pts values was selected for testing. We then determined values for $Epsilon$ by adapting the "elbow method" used for determining the best value of k in k-Means clustering [2]. Figure 2 shows the sorted Cosine distance to the nearest neighbor for each participant. We propose that ideal values for $Epsilon$ will be around the elbow of this figure. In the case of figure 2, this range is [0.15, 0.35].

$$Cosine_distance(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|} \quad (1)$$

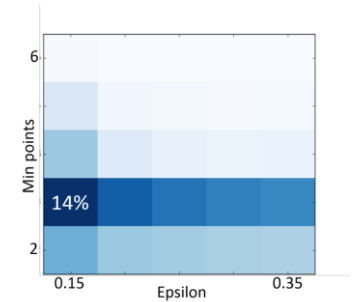


Figure 3: Information gain scoring method for Cosine distance metric across several $Epsilon$ and Min_pts values.

3.4 Evaluation

To evaluate DBSCAN performance, we developed a scoring metric based on information gain. This scoring metric, seen in equation 2, considers the amount of information gain and size of each cluster in order to produce a value between 0 and 1. We scored each set of DBSCAN parameters and compared scores to find the best settings.

$$score = \frac{ClusteredPatients}{TotalPatients} \cdot \frac{InformationGain}{MaxInformationGain} \quad (2)$$

4. RESULTS

We observed that Cosine distance scored much higher than Euclidean distance. Figure 3 shows the scores of our parameter sweep around initial parameter selections for the Cosine distance metric. There is a clear set of parameters at $Epsilon = 0.15$ and $Min_pts = 3$ which provides the best clustering score. The maximum observed score from our testing was 14%. Observed scores from our clustering were unexpectedly low. Despite the poor ability of DBSCAN to differentiate between patients with and without diabetes, our method of focused parameter searching was able to find optimal parameter settings.

5. REFERENCES

- [1] K. Doi. Computer-aided diagnosis in medical imaging: historical review, current status and future potential. *Computerized medical imaging and graphics*, 31(4):198–211, 2007.
- [2] T. M. Kodinariya and P. R. Makwana. Review on determining number of cluster in k-means clustering. *International Journal*, 1(6):90–95, 2013.
- [3] I. Kononenko. Machine learning for medical diagnosis: history, state of the art and perspective. *Artificial Intelligence in medicine*, 23(1):89–109, 2001.

Project: Extended Abstracts

- Dylan Chapp and Surya Kasturi. Leveraging Spark and Docker for Scalable, Reproducible Analysis of Railroad Defects
- Stephen Herbein and Sean McAllister. Clustering Temporal Gene Expressions of Iron-Oxidizing Zetaproteobacteria
- Moumita Bhattacharya and Debarati Roychowdhury. Using Machine Learning to Build a Scalable Tool to support Dietitians to Fight Chronic Diseases

Project: Extended Abstracts

- Paul Soper Validation of the Short Time-series Expression Miner (STEM) on Iron Cycling in a Shallow Alluvial Aquifer
- Michael Wyatt. Parameter Tuning of DBSCAN for Medical Data and Diabetes Diagnosis

Select three (3) projects from the five (5) listed above and made available in Lecture 05

Project: Analysis of Abstracts

For the three projects you have selected:

- Read the extended abstracts
- Identify:
 - Type of data
 - Question(s) answered
 - Methods used
 - Key outcomes/results
- Identify the parts with:
 - Problem definition; Methodology to resolve the problem; Outcome or results

Project: Analysis of Posters

For the three projects you have selected:

- Carefully inspect the posters
- Identify:
 - Type of data
 - Question(s) answered
 - Methods used
 - Key outcomes/results
- Identify the parts with:
 - Problem definition; Methodology to resolve the problem; Outcome or results

Project: **Abstract vs. Poster**

Write a short report (600 words) answering these questions:

- What are the common components of both abstracts and posters?
- What are the key differences?
- What is the role of pictures in abstracts and posters?
Compare and contrast.
- What is the role of text in abstracts and posters?
Compare and contrast.

Project: Report your Findings

- Write a short report to summarize your findings (no more than 600 words) in the Assignment 05 notebook



THE UNIVERSITY OF
TENNESSEE
KNOXVILLE
BIG ORANGE. BIG IDEAS.®

